1️⃣ np.where()

🔹 Definition

Returns indices where condition is True
OR
Returns values based on condition (vectorized if-else)

✅ Form 1 — Get indices

np.where(condition)

Example:

a = np.array([10, 20, 30, 40])
np.where(a > 20)

Output:

(array([2, 3]),)

⚠ Always returns a tuple of arrays.

For 2D:

np.where(a > 5)

Returns:

(row_indices, col_indices)

✅ Form 2 — Vectorized if-else

np.where(condition, x, y)

Meaning:

if condition → take x
else → take y

Example:

np.where(a % 2 == 0, a, -1)

2️⃣ np.searchsorted()

🔹 Definition

Finds insertion index in a **sorted array**.

⚠ Array MUST be sorted.

Syntax

np.searchsorted(sorted_array, value)

Example:

a = np.array([10,20,30,40])
np.searchsorted(a, 25)

Output:

2

Because 25 should go before 30.

Side parameter

np.searchsorted(a, 20, side='left')
np.searchsorted(a, 20, side='right')

- 'left' → first occurrence
- 'right' → after last occurrence

Professional use: binary search operations.

print(np.sort(a6,axis=1))

3️⃣ np.unique()

🔹 Definition

Returns sorted unique values.

np.unique(a)

⚠ Always sorted.

Important Parameters

np.unique(a, return_counts=True)
np.unique(a, return_index=True)
np.unique(a, return_inverse=True)

return_counts

Gives frequency (very important for ML).

Example:

values, counts = np.unique(a, return_counts=True)

4️⃣ np.sort()

🔹 Definition

Returns sorted copy.

np.sort(a)

⚠ Does NOT modify original.

Axis behavior

For 2D:

```
np.sort(a, axis=0)  # column-wise
np.sort(a, axis=1)  # row-wise
```

Default:

axis = -1 (last axis)

Professional note:
Use a.sort() if you want in-place sorting.

5️⃣ np.argsort()

🔹 Definition

Returns indices that would sort the array.

Very important.

Example:

```
a = np.array([40,10,30])
np.argsort(a)
```

Output:

[1,2,0]

Meaning:
Sorted order is:

a[1], a[2], a[0]

Powerful Use

Sort another array based on this:

sorted_a = a[np.argsort(a)]

Or sorting by column in 2D:

a[a[:,1].argsort()]

Very important interview concept.

6️⃣ np.partition()

🔹 Definition

Partially sorts array.

It places the kth element in correct position
Everything smaller → left
Everything larger → right
But not fully sorted.

Example:

a = np.array([7,2,1,6,8])
np.partition(a, 2)

The element at index 2 will be correct if sorted.

⚠ Faster than full sort for top-k problems.

Used in:

- Finding median
- Top k smallest/largest

7️⃣ Boolean Filtering Mastery

This is essential.

🔹 Basic Filtering

a[a > 10]

Returns elements > 10.

🔹 Multiple Conditions

Use & and |

a[(a > 10) & (a < 30)]

⚠ MUST use parentheses.

Wrong:

a[a > 10 & a < 30] ❌

🔹 2D Filtering

a[a > 5]

Flattens result.

If you want structure:
Use mask directly:

mask = a > 5

🔹 Replace Using Boolean

a[a % 2 == 0] = 0

Professional-level operation.

🔥 CRITICAL DIFFERENCES (Memorize This Table)

Function	Returns	Modifies Original	Key Use
Where	indices or values	No	conditional logic
searchsorted	insertion index	No	binary search
unique	sorted unique values	No	frequency
Sort	sorted copy	No	full sorting

Function	Returns	Modifies Original	Key Use
argsort	indices	No	indirect sorting
partition	partial sort	No	kth smallest
boolean filtering	filtered values	Yes (if assigned)	data cleaning

---

🔥 **Interview-Level Concepts**

1. argsort is more powerful than sort.
2. partition is O(n), sort is O(n log n).
3. unique always sorts output.
4. where returns tuple for multidimensional arrays.
5. Boolean masks must match array shape.